



كلية علوم الحاسب والمعلومات
قسم تقنية المعلومات

King Saud University College of Computer and Information Sciences
Department of Information Technology
IT462: Big Data Systems
2nd Semester 1447 H



Phase 6: Final Report

LinkedIn Job Market Intelligence

Section #: 72889 Group #: 1	NAME	ID	Email
	Rama Khalid Alomair	444200662	444200662@student.ksu.edu.sa
	Walaa Saifaleslam	444200088	444200088@student.ksu.edu.sa
	Mona Saleh Alnajjar	444200091	444200091@student.ksu.edu.sa
	Alya Saud Althobiani	444200973	444200973@student.ksu.edu.sa

Supervised By: Dr. Afshan Jafri

Table of Contents

1. Introduction.....	5
1.1 Background.....	5
1.2 Problem Statement	5
1.3 Motivation	5
1.4 Objectives	6
1.5 Scope	6
2. Data selection and understanding.....	6
2.1 Dataset Description.....	6
2.2 Dataset Size and Format	6
2.3 Main Attributes	7
2.4 Initial Observation.....	7
2.5 Ethical and Legal Considerations	8
3. Data preprocessing.....	8
3.1 Cleaning.....	8
3.2 Integration.....	9
3.3 Reduction.....	10
3.4 Transformation.....	10
3.5 Summary of Preprocessing Pipeline	12
4. RDD operations.....	12
4.1 RDD Design.....	12
Data Model at the RDD Level	12
4.2 Key Transformations and Actions	13
Analysis 1 — Remote Job Filtering	13
Analysis 2 — Salary Tier Mapping.....	13
Analysis 3 — Experience-Level Grouping and Maximum Salary Detection	14
Analysis 4 — Average Salary by State	15
Analysis 5 — Ranking State by Salary	15
4.3 Reflections on RDD Usage	16
Performance and Scalability Observations.....	16
Limitations of RDD Usage.....	16
5. SQL operations.....	16
5.1 DataFrame and View Creation	16
5.2 Key Analytical Queries	16
Query 1 — Average Salary by Experience Level	16
Query 2 — Top States by Average Salary	17
Query 3 — Salary Ranking within Experience Levels	17
Query 4 — Jobs Above State Average Salary	18
Query 5 — Salary Distribution by Work Type	18

5.3	Summary of Insights from SQL	19
6.	Machine learning operations	19
6.1	ML Problem Formulation.....	19
6.2	Feature Engineering & Data Preparation.....	20
6.3	Model Choice	20
6.4	Training and Evaluation Setup	21
6.5	Results.....	21
6.6	Interpretation.....	21
6.7	Limitations	22
7.	Conclusion and future work	22
7.1	Summary of contribution.....	22
7.2	Reflection.....	22
7.3	Potential Extensions (Future Work).....	22
8.	References	24
9.	Appendix	25
	Appendix A: RDD Transformation Snippet	25
	Appendix B: SQL Query Snippet.....	29
	Appendix C: Machine Learning Operations Snippet.....	32

Table of Tables

Table 1:	Dataset Size and Format	6
Table 2:	Main Attributes	7
Table 3:	Cleaning Summary	9
Table 4:	Reduction Summary	10
Table 5:	Engineered Features.....	11
Table 6	Salary Tier Mapping Sample Output	14
Table 7	Average Salary by State Sample Output.....	15
Table 8	Ranking State by Salary Sample Output.....	16
Table 9	Average Salary by Experience Level Sample Output	17
Table 10	Top States by Average Salary Sample Output.....	17
Table 11	Salary Ranking within Experience Levels Sample Output.....	18
Table 12	Jobs Above State Average Salary Sample Output.....	18
Table 13	Salary Distribution by Work Type Sample Output	19
Table 14:	Features used in the ML Model	20
Table 15:	Feature Selection Decisions.....	20
Table 16:	Key Hyperparameters	21
Table 17:	Models Results.....	21

Table of Figures

Figure 1: Preprocessing Pipeline	12
Figure 2: Remote Job Filtering Snippet.....	13
Figure 3 Salary Tier Mapping Snippet.....	13
Figure 4 : Experience Level Grouping and Maximum Salary Detection Snippet	14
Figure 5 Average Salary by State Snippet	15
Figure 6 Ranking State by Salary Snippet	15

1. Introduction

1.1 Background

The modern job market generates massive volumes of data daily through online platforms such as LinkedIn, which hosts millions of job postings across industries, roles, and geographies. This data contains rich information about hiring trends, required skills, salary ranges, and work arrangements. However, extracting meaningful insights from such large-scale, heterogeneous datasets requires advanced data processing and analytical techniques beyond traditional tools.

Apache Spark has emerged as the leading framework for big data processing, offering distributed computing, in-memory processing, and a unified API for batch analytics, SQL queries, and machine learning. This project leverages Spark to analyze a real-world LinkedIn job postings dataset and build a salary prediction model.

1.2 Problem Statement

Despite the availability of large-scale job posting data on platforms such as LinkedIn, there is a significant gap in transforming this raw information into actionable insights. Job seekers struggle to understand which skills are in demand, how requirements vary across industries and locations, and what factors drive salary differences. Educational institutions and policymakers similarly lack tools to evaluate labor market trends and align academic programs with industry needs.

The core problem addressed in this project is: **how to effectively analyze large-scale LinkedIn job postings to uncover job market trends and develop accurate salary prediction models** based on job characteristics such as role, required skills, experience level, and location. This analysis is relevant to multiple stakeholders: job seekers seeking salary transparency, educational institutions aligning curricula with market demands, employers benchmarking compensation, and policymakers monitoring labor market trends.

1.3 Motivation

The primary motivation is the growing need for data-driven insights into today's labor market. With over 123,000 job postings available, this dataset provides a valuable opportunity to apply big data and machine learning techniques to a problem with real social and economic impact. Specifically, this project aims to:

- Help students and job seekers understand current market demands and align their learning paths accordingly.
- Support educational institutions in adapting curricula to reflect real industry needs.
- Assist employers and policymakers with evidence-based insights into hiring trends and salary patterns.
- Serve as a practical application of large-scale data processing, exploratory analysis, predictive modeling, and performance evaluation.

1.4 Objectives

The main objectives of this project are to:

1. Perform exploratory data analysis on LinkedIn job postings using Spark RDD and SQL operations.
2. Preprocess and clean the dataset to ensure data quality and readiness for modeling.
3. Identify key factors influencing salary levels across roles, locations, and work types.
4. Build and evaluate regression models to predict normalized salary.

1.5 Scope

This project focuses on a single LinkedIn job postings dataset of approximately 123,392 records after preprocessing. The analysis is conducted entirely within the Apache Spark framework using Scala. The prediction task is limited to salary regression using structured job attributes; unstructured text fields such as full job descriptions are excluded from modeling. Out of scope includes real-time job posting analysis, deep hyperparameter tuning, and external data sources beyond the provided dataset. Additionally, the dataset is limited to U.S.-based job postings and may not generalize to other labor markets.

2. Data selection and understanding

2.1 Dataset Description

The dataset used in this project is the **LinkedIn Job Listings** dataset, sourced from Hugging Face, a widely used platform for open-access machine learning datasets [1]. It belongs to the labor market and recruitment domain, commonly used for job market analysis, recommendation systems, text mining, and workforce research. LinkedIn, as one of the world's largest professional networking platforms, generates large volumes of structured job posting data that reflect real hiring activity across industries and geographies, making this dataset a representative and practical resource for labor market analysis.

2.2 Dataset Size and Format

job_id serves as the primary key for this dataset, uniquely identifying each job posting. No foreign keys or relational links to external tables exist, as the dataset consists of a single CSV file.

Table 1: Dataset Size and Format

Property	Value
Number of rows (raw)	~123,849
Number of rows (after cleaning)	123,392
File size	~ 494 MB
Format	Single CSV file
Number of attributes	34 (raw), 32 (after preprocessing)

Time coverage	March – April 2024
---------------	--------------------

2.3 Main Attributes

Table 2: Main Attributes

Attribute	Type	Description	Example value
job_id	int64	Unique job posting identifier	921716
company_name	string	Name of the hiring company	Corcoran Sawyer Smith
title	string	Job title	Marketing Coordinator
location	string	Job location (city/state/region)	Princeton, NJ
normalized_salary	float64	Target variable – standardized annual salary	38,480.0
min_salary / max_salary	float64	Salary range bounds	17.0 / 20.0
formatted_experience_level	string	Seniority (Entry, Mid-Senior, Director, etc.)	Mid-Senior level
formatted_work_type	string	Work arrangement (Full-time, Contract, Internship, etc.)	Full-time
remote_allowed	string	Whether remote work is permitted	0 (Not remote)
description	string	Full job description text	"A leading real estate firm..."
skills_desc	string	Required skills (high missingness – later dropped)	NaN
sponsored	int64	Whether the posting is sponsored	0
views / applies	float64	Engagement metrics	20.0 / 2.0

2.4 Initial Observation

An initial review of the dataset revealed several notable characteristics:

- **High missingness in key columns:** closed_time (99.13%), skills_desc (98.03%), and med_salary (94.93%) were nearly empty, rendering them unusable for analysis.
- **Salary inconsistencies:** Some records contained missing, zero, or logically inconsistent salary values (min > max), requiring careful handling.
- **High cardinality:** The title and location columns contained thousands of unique values, introducing dimensionality challenges for modeling.

- **Timestamp format:** Time-related columns were stored in Unix millisecond format rather than readable timestamps.
- **Job type distribution:** Full-time postings dominate the dataset with 98,814 records, while internships account for only 983.
- **Geographic distribution:** Postings span across U.S. states and metropolitan areas, with technology hubs such as San Francisco, New York, and Seattle showing the highest salary averages.
- **Remote work:** Only 12.3% of postings are remote, indicating that on-site and hybrid roles dominate the dataset.

These initial observations directly informed the preprocessing decisions taken in Phase 2 and highlighted which features carry the most predictive signal for salary estimation.

2.5 Ethical and Legal Considerations

The dataset used in this project was obtained from Hugging Face, a publicly accessible platform for open-source machine learning datasets. It is made available for academic and research purposes. The dataset does not contain direct personal identifiers such as applicant names, national IDs, or contact information, as it consists solely of job posting metadata published publicly on LinkedIn. No data collection, scraping, or privacy violations were involved in obtaining this dataset. Its use in this project is limited to educational analysis within the IT 462 course.

3. Data preprocessing

The LinkedIn Job Listings dataset required several preprocessing steps before it could be reliably used for Spark RDD operations, SQL analytics, and machine learning. The preprocessing workflow focused on improving data quality, reducing dimensionality, transforming categorical variables into machine-readable representations, and preparing the dataset for salary prediction tasks. The preprocessing pipeline was implemented entirely in Apache Spark using Scala and organized into four main stages: Cleaning, Integration, Reduction, and Transformation.

3.1 Cleaning

The raw dataset initially contained 123,849 rows and 34 columns. During exploratory analysis, several data quality issues were identified, particularly extensive missing values across multiple attributes. The columns **closed_time** (99.13% missing), **skills_desc** (98.03% missing), and **med_salary** (94.93% missing) were almost entirely sparse and therefore removed completely because reliable imputation was not possible and these fields contributed limited analytical value.

Other attributes contained partial missing values and were handled differently depending on the variable type.

Categorical attributes such as **formatted_experience_level**, **formatted_work_type**, **company_name**, **pay_period**, **currency**, **compensation_type**, **application_url**, and **posting_domain** were filled using the placeholder category "Unknown" to preserve records while explicitly indicating unavailable information.

Numeric attributes including **remote_allowed**, **views**, **applies**, **zip_code**, **fips**, and **company_id** were imputed using 0.0 when the missing values represented unavailable metadata rather than meaningful measurements.

Salary-related variables required more careful handling because they represent the target variable for the prediction model. Missing values in **normalized_salary**, **min_salary**, and **max_salary** were imputed using median values rather than means because salary distributions are highly skewed and contain extreme outliers. Median imputation therefore provides more robust and representative estimates. The computed medians were:

normalized_salary = 82,500, **min_salary** = 62,000 and **max_salary** = 82,000.

Additionally, a new feature named **salary_source** was introduced to distinguish between Original and Imputed salary values. This allowed later machine learning stages to exclude imputed salary targets while still preserving all records for RDD and SQL analysis.

Several validation checks were also applied to ensure logical consistency in salary-related fields. Salary outliers were removed by retaining only records where **normalized_salary** was between \$10,000 and \$1,000,000 or originally null before imputation. This filtering removed unrealistic compensation values while preserving valid market variation. A total of 455 rows were removed during this process.

A sanity check was also performed to detect records where **min_salary** > **max_salary**. No invalid records of this type were found.

Duplicate detection was performed using the **job_id** field as a unique identifier. The dataset contained 123,849 distinct job IDs and no duplicate records. Nevertheless, a deduplication step using `dropDuplicates("job_id")` was still applied as a safeguard to ensure dataset integrity.

Timestamp-related attributes originally stored in Unix millisecond format were converted into readable Spark TimestampType representations, including **listed_time_ts**, **original_listed_time_ts**, and **expiry_ts**. This improved readability and enabled future temporal analysis. Timestamp fields were standardized into consistent Spark TimestampType representations to improve temporal consistency during analysis.

Table 3: Cleaning Summary

Issue	Before	After
Columns with > 94% missing values	3	0
Salary outlier rows	455	0
Duplicate job IDs	0	0
Rows with min_salary > max_salary	0	0
Dataset rows	123,849	123,392
Dataset columns	34	32

After cleaning, the dataset contained 123,392 rows and 32 columns ready for downstream processing.

3.2 Integration

No external data integration was required in this project because the dataset consisted of a single CSV file (`linkedin_jobs.csv`). Therefore, no joins, schema merging, or conflict resolution procedures were necessary. All preprocessing operations were applied directly to the single-source dataset.

The final analytical schema after preprocessing retained the primary attributes required for salary prediction and analytics:

title, **state**, **formatted_work_type**, **formatted_experience_level**, **normalized_salary**, **salary_source** and **remote_allowed**.

Additional indexed and encoded representations of these variables were later generated during the transformation stage.

3.3 Reduction

After cleaning, the dataset underwent a structured reduction phase to lower dimensionality and retain only features relevant to salary prediction. Sampling was not used because the final dataset size (123,392 rows) was manageable within the Spark environment, and retaining all records preserved salary variability and rare job patterns important for regression modeling.

Similarly, aggregation was intentionally avoided because the project objective involves row-level salary prediction. Aggregating records into grouped summaries would remove important job-level detail required for machine learning and salary analysis.

Several non-informative or redundant attributes were removed during feature selection, including URLs, technical identifiers, timestamp metadata, and administrative fields. Examples of removed columns include: **job_posting_url**, **application_url**, **posting_domain**, **expiry**, **listed_time**, **listed_time_ts**, **expiry_ts**, **company_id**, **zip_code**, **fips** and **description**

These attributes were excluded because they either introduced noise, increased dimensionality, or provided limited predictive value for salary estimation.

Redundant salary attributes (**min_salary** and **max_salary**) were also removed to avoid multicollinearity while retaining **normalized_salary** as the primary regression target.

The original location column contained highly granular city-level values with extremely high cardinality. To reduce categorical complexity, the state component was extracted from the location string and stored in a new feature named **state**, after which the original location field was removed.

Table 4: Reduction Summary

Metric	Before	After
Rows	123,392	123,392
Columns	32	7

The reduction phase significantly decreased dimensionality while preserving all records required for RDD operations, SQL analytics, and machine learning.

3.4 Transformation

The transformation phase converted the reduced dataset into a machine-learning-ready representation suitable for Spark ML pipelines.

Additional TimestampType columns were derived from Unix millisecond timestamp fields to improve temporal readability and consistency during analysis. In addition, multiple features were explicitly cast into numeric types before model training, including **normalized_salary**, **state_idx_num**, **formatted_work_type_idx_num**, **title_idx_num**, **formatted_experience_level_idx_num**, and **remote_allowed**.

During preprocessing, categorical text variables were transformed into numerical index-based representations using Spark ML's StringIndexer in order to make them compatible with downstream machine learning operations.

The following categorical columns were indexed:

title, **state**, **formatted_work_type**, **formatted_experience_level** and **salary_source**

This transformation produced the following indexed numerical features:

title_idx_num, **state_idx_num,** **formatted_work_type_idx_num,**
formatted_experience_level_idx_num and **salary_source_idx_num**

All StringIndexer stages were configured using `handleInvalid = "keep"` to safely process unseen or invalid categories during transformation.

For the title column, rare job titles appearing fewer than five times were grouped under the label "Other" before indexing in order to reduce sparsity and excessive dimensionality.

During the machine learning stage, nominal categorical variables (**state**, **formatted_work_type**, and **title**) were further transformed using OneHotEncoder. This converted indexed categories into binary vectors so the model would not assume ordinal relationships between unrelated categories. One-Hot Encoding was applied only during the machine learning stage rather than earlier preprocessing stages to minimize intermediate dataset dimensionality.

The feature **formatted_experience_level** remained numerically indexed because experience levels naturally represent ordinal progression.

A VectorAssembler then combined all transformed features into a single vector named **features_raw**. Afterwards, a StandardScaler was applied with both mean centering and standard deviation scaling enabled, producing the final normalized feature vector named **features**. This ensured that variables with larger numeric ranges would not dominate the learning process.

Table 5: Engineered Features

Feature	Purpose
salary_source	Distinguishes original vs imputed salaries
state	Reduces location cardinality
title_idx_num	Encoded representation of job titles
state_idx_num	Encoded representation of states
formatted_work_type_idx_num	Encoded work arrangement
formatted_experience_level_idx_num	Encoded seniority level

These engineered features were designed to preserve meaningful labor-market characteristics while improving compatibility with Spark ML algorithms and reducing categorical complexity.

To prevent data leakage, the dataset was split into 80% training data and 20% testing data before scaling and transformation operations. Additionally, only rows where **salary_source** = Original were retained during machine learning training to avoid bias caused by median-imputed target values.

3.5 Summary of Preprocessing Pipeline

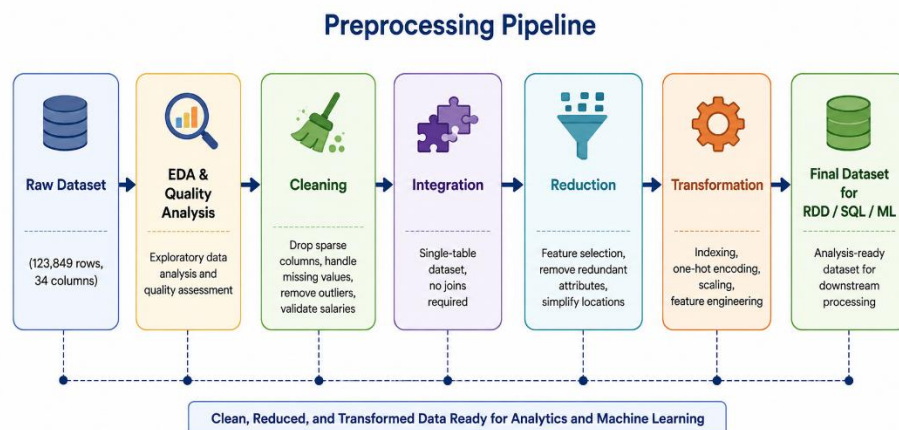


Figure 1: Preprocessing Pipeline

The preprocessing implementation was primarily developed in:

- 01_DataPreprocessing.scala
- 04_MachineLearning.scala

The generated datasets were saved as:

- preprocessed_dataset.csv

4. RDD operations

4.1 RDD Design

Data Model at the RDD Level

The dataset was loaded into Spark as a raw text RDD using `textFile()`, then transformed into structured RDDs through splitting and mapping operations. The main RDD structure used throughout the analysis was:

`RDD[(state, (salary, experienceLevel, workType, title))]`

Additional derived RDD structures included:

- (state, salaryTier, workType, salary)
- (experienceLevel, title)
- (state, (salary, 1))
- (averageSalary, state)

These structures enabled flexible distributed transformations and aggregations across the LinkedIn jobs dataset.

RDDs were useful in this project because they provided:

- Fine-grained control over transformations and parsing logic
- Flexible handling of irregular CSV rows and quoted fields
- Ability to manually engineer features such as salary tiers
- Easier demonstration of core distributed computing concepts

- Explicit transformation and action workflow for educational analysis

Unlike DataFrames, RDDs allowed custom Scala functions such as `safeGet()`, `parseDouble()`, and `cleanExperienceLevel()` to directly manipulate records at a lower abstraction level.

4.2 Key Transformations and Actions

Analysis 1 — Remote Job Filtering

Purpose: Compute the number and percentage of remote jobs in the dataset.

Inputs: Raw LinkedIn job records and `remote_allowed` column

Operations Used : “filter”, “count”, “first”

Scala Snippet

```
val remoteJobsRDD = splitRDD.filter(row =>
  | safeGet(row, remoteIndex, "0.0") == "1.0"
  | )
val remoteCount = remoteJobsRDD.count()
```

Figure 2: Remote Job Filtering Snippet

Sample Output

- Total jobs: 123,392
- Remote jobs: 15,168
- Remote percentage: 12.3%

Interpretation

This analysis answers the question: How common are remote jobs? The results showed that only a small percentage of jobs supported remote work, indicating that most positions still required physical presence. This provided insight into work arrangement patterns before salary modeling.

Analysis 2 — Salary Tier Mapping

Purpose: Transform raw salary values into salary categories for easier comparison.

Inputs: “normalized_salary”, “state”, “formatted_work_type”

Operations Used : “map”, “take”

Scala Snippet

```
val mappedRDD = splitRDD.map(row => {
  val salary = parseDouble(safeGet(row, salaryIndex, "0.0"))

  val salaryTier =
    if (salary < 60000) "Low"
    else if (salary < 100000) "Mid"
    else "High"

  (state, salaryTier, workType, salary)
})
```

Figure 3 Salary Tier Mapping Snippet

Sample Output:

Table 6 Salary Tier Mapping Sample Output

State	Tier	Salary
NC	Low	35,360
TX	Mid	82,500
NY	Low	53,000

Interpretation:

This analysis answers the question: *How can salary values be simplified for comparison?* The mapping transformation created interpretable salary groups. Most internship positions appeared in the Low tier, while many full-time positions appeared in the Mid tier. This simplified salary pattern analysis across locations and job types.

Analysis 3 — Experience-Level Grouping and Maximum Salary Detection

Purpose: Analyze how job titles are distributed across experience levels and identify the highest-paying record.

Inputs: “formatted_experience_level”, “title”, “normalized_salary”

Operations Used : “groupByKey”, “reduce”

Scala Snippet

```
val titlesByExperienceRDD =  
  splitRDD.map(row => (experience, title)).groupByKey()  
  
val maxSalaryRecord =  
  salaryRecordRDD.reduce((a, b) =>  
    if (a._1 >= b._1) a else b  
  )
```

Figure 4 : Experience Level Grouping and Maximum Salary Detection Snippet

Sample Output:

- Entry level: 21,494 unique titles
- Mid-Senior level: 26,079 unique titles
- Maximum salary: \$960,000

Interpretation:

This analysis answers the question: *How does experience level relate to salary and job variety?* The results showed that Mid-Senior level positions had the highest number of unique job titles, suggesting greater diversity in experienced roles. The maximum salary value of \$960,000 appeared as a significant outlier, revealing extreme compensation variation within the dataset.

Analysis 4 — Average Salary by State

Purpose: Compute average normalized salary for each state.

Inputs: “state”, “normalized_salary”

Operations Used : “reduceByKey”, “mapValues”, “saveAsTextFile”

Scala Snippet

```
val stateTotalsRDD =  
  stateSalaryRDD.reduceByKey((a, b) =>  
    (a._1 + b._1, a._2 + b._2)  
  )  
  
val stateAvgRDD =  
  stateTotalsRDD.mapValues {  
    case (total, count) => total / count  
  }
```

Figure 5 Average Salary by State Snippet

Sample Output:

Table 7 Average Salary by State Sample Output

State	Average Salary
Massachusetts	80,826
Louisiana Metropolitan Area	107,778

Interpretation:

This analysis answers the question: *How does salary vary geographically?* The results revealed strong salary differences across locations. Metropolitan regions generally showed higher compensation levels, confirming that location is an important feature for salary prediction.

Analysis 5 — Ranking State by Salary

Purpose: Rank locations according to average salary.

Inputs: average salary per state.

Operations Used : “sortByKey”, “saveAsTextFile”

Scala Snippet

```
val sortedRDD = stateAvgRDD  
  .map { case (state, avg) => (avg, state) }  
  .sortByKey(false)
```

Figure 6 Ranking State by Salary Snippet

Sample output:

Table 8 Ranking State by Salary Sample Output

State	Average Salary
Illinois	278,750
San Francisco Bay Area	119,270

Interpretation:

This analysis answers the question: Which locations offer the highest salaries? The results showed that major metropolitan regions consistently ranked among the highest-paying areas. Illinois appeared as an unusually high-paying location, likely influenced by salary outliers.

4.3 Reflections on RDD Usage

Performance and Scalability Observations

- Spark distributed transformations efficiently across partitions.
- `reduceByKey()` improved scalability by reducing shuffle overhead.
- `coalesce(1)` was used to generate a single clean CSV output file.
- RDD lineage supported fault tolerance throughout execution.

Limitations of RDD Usage

- RDD code was more verbose than DataFrame-based implementations.
- Manual CSV parsing increased implementation complexity.
- Nested tuple structures made debugging harder.
- `groupByKey()` may consume large memory resources on big datasets.

5. SQL operations

5.1 DataFrame and View Creation

The preprocessed LinkedIn jobs dataset was loaded into Spark using the DataFrame API with schema inference enabled. CSV options such as header, multiLine, quote, and escape were used to correctly handle quoted values and multiline records. After loading, the dataset was registered as a temporary SQL view named **jobs** to enable SQL-based analysis.

Dataset summary:

- Rows: 123,392
- Columns: 12
- Temporary SQL view: jobs

5.2 Key Analytical Queries

Query 1 — Average Salary by Experience Level

Natural Language Question: What is the average salary for each experience level with more than 100 jobs posting?

SQL Query:

```
SELECT formatted_experience_level,  
ROUND(AVG (normalized_salary),2) AS avg_salary,  
COUNT (*) AS job_count
```

```
FROM jobs
GROUP BY formatted_experience_level
HAVING COUNT (*) > 100
ORDER BY avg_salary DESC
```

Sample Result:

Table 9 Average Salary by Experience Level Sample Output

Experience Level	Avg Salary
Executive	119,887
Director	113,012
Mid-Senior level	92,308

Interpretation:

Higher experienced levels consistently showed larger average salaries, confirming that seniority strongly affects compensations.

Query 2 — Top States by Average Salary

Natural Language Question: Which states have the highest average salaries?

SQL Query:

```
SELECT state,
ROUND(AVG (normalized_salary),2) AS avg_salary
FROM jobs
GROUP BY state
HAVING COUNT (*) > 50
ORDER BY avg_salary DESC
LIMIT 10
```

Sample Result:

Table 10 Top States by Average Salary Sample Output

State	Avg Salary
San Francisco Bay Area	119,269
New York City Metropolitan Area	106,076
Greater Seattle Area	105,692

Interpretation:

Major metropolitan regions offered the highest salaries, showing that location is an important predictor of compensation.

Query 3 — Salary Ranking within Experience Levels

Natural Language Question: How does each job rank in salary within its experience level?

SQL Query:

```
SELECT title,
formatted_experience_level,
normalized_salary,
RANK () OVER (
PARTITION BY formatted_experience_level
```

```
ORDER BY normalized_salary DESC
) AS salary_rank
FROM jobs
```

Sample Result:

Table 11 Salary Ranking within Experience Levels Sample Output

Title	Salary	Rank
Modernization Software Dev	960,000	1
Structured Settlement Account Manager	435,000	2

Interpretation:

The window function revealed strong salary variation within the same experience level and highlighted extreme salary outliers.

Query 4 — Jobs Above State Average Salary

Natural Language Question: Which jobs pay above their state's average salary?

SQL Query:

```
WITH state_avg AS (
SELECT state,
AVG(normalized_salary) AS avg_state_salary
FROM jobs
GROUP BY state
)
SELECT j.title, j.state,
j.normalized_salary
FROM jobs j
JOIN state_avg sa
ON j.state = sa.state
WHERE j.normalized_salary > sa.avg_state_salary
```

Sample Result:

Table 12 Jobs Above State Average Salary Sample Output

Title	State	Salary
Modernization Software Dev	WI	960,000
Quantitative Researcher	Greater Chicago Area	950,000

Interpretation:

The query identified jobs significantly exceeding regional averages, revealing high paying specialized positions and strong salary outliers.

Query 5 — Salary Distribution by Work Type

Natural Language Question: How does salary distribution vary across work types?

SQL Query:

```
SELECT formatted_work_type,
```

```

AVG (normalized_salary),
STDDEV_POP (normalized_salary),
PERCENTILE_APPROX (normalized_salary,0.50)
FROM jobs
GROUP BY formatted_work_type

```

Sample Result:

Table 13 Salary Distribution by Work Type Sample Output

Work Type	Avg Salary
Contract	88,885
Full-time	87,432
Internship	75,852

Interpretation:

Contract and full-time jobs generally offered higher salaries than internships and part-time positions, indicating that work arrangement influences compensation.

5.3 Summary of Insights from SQL

- Experience level strongly influenced salary, with Executive and Director roles having the highest average compensation.
- Geographic location significantly affected salaries, especially in major metropolitan areas such as San Francisco and New York.
- Window functions revealed large salary variation within the same experience level.
- Several extreme salary outliers were identified through ranking and comparison queries.
- Work type impacted salary distribution, with contract and full-time roles generally paying more than internships and part-time positions.

6. Machine learning operations

6.1 ML Problem Formulation

The goal of this phase is to predict the **annualized salary** (Target Variable: `normalized_salary`) of LinkedIn job postings based on several descriptive features. This is a **Supervised Regression** problem, where we aim to minimize the error between the predicted and actual salary values.

Input Features: Job title, state, work type, experience level, and remote allowance were selected as predictors based on their demonstrated correlation with salary in the RDD and SQL analysis phases.

Constraints:

- **Noisy Labels:** Salary values exhibit high variance within the same job title and location, introducing noise into the target variable.
- **Data Filtering:** Only records with original (non-imputed) salary values were retained for training to ensure ground-truth accuracy, reducing the training set size.
- **High Cardinality:** The title and state columns contained thousands of unique values, requiring careful encoding to avoid excessive dimensionality.

6.2 Feature Engineering & Data Preparation

To ensure high-quality input for the model, we implemented a structured pipeline for data preparation:

- **Data Filtering:** We restricted the dataset to "Original" salary sources only, removing imputed or estimated values to ensure ground-truth accuracy.
- **Type Casting:** All features were cast to Double to maintain precision and compatibility with Spark ML stages.
- **Pipeline Transformations:**
 1. **One-Hot Encoding:** Categorical indices (State, Work Type, Title) were converted into binary vectors.
 2. **Vector Assembler:** All numerical and encoded features were consolidated into a single feature vector.
 3. **Standard Scaling:** Features were normalized (Mean=0, Std=1) to prevent features with larger ranges from dominating the model.

Features used in the ML Model:

Table 14: Features used in the ML Model

Feature Type	Variables Included	Transformation
Categorical	State, Job Title, Work Type	One-Hot Encoding
Numerical	Experience Level Index	Standard Scaling
Binary	Remote Allowed (0/1)	None

Table 15: Feature Selection Decisions

Feature	Decision	Reason
State	Included	Strong geographic salary signal confirmed in SQL analysis
Job Title	Included	Role directly impacts compensation
Work Type	Included	Contract vs Full-time affects salary
Experience Level	Included	Seniority strongly correlates with salary
Remote Allowed	Included	Work arrangement influences compensation
Description	Excluded	Unstructured text, out of scope
Skills	Excluded	98% missing values
Min/Max Salary	Excluded	Would cause data leakage

6.3 Model Choice

Two ensemble-based regression models were selected:

Random Forest Regressor was chosen for its ability to handle non-linear relationships, reduce overfitting through bagging, and provide feature importance scores for interpretation.

Gradient-Boosted Trees (GBT) was chosen as it iteratively corrects errors from previous trees, making it more powerful for complex salary patterns.

Key Hyperparameters:

Table 16: Key Hyperparameters

Hyperparameter	Random Forest	GBT
Number of Trees	50	—
Max Depth	10	5
Max Iterations	—	100
Seed	42	42

6.4 Training and Evaluation Setup

- Train/Test split: **80% / 20%**, random seed = **42**
- Only records with original salary values were used for training
- Evaluation metrics: **RMSE**, **MAE**, **R²** — appropriate for regression tasks

6.5 Results

Table 17: Models Results

Model	RMSE	MAE	R ²
Baseline (Mean)	\$62,016	\$44,405	-0.0005
Random Forest	\$53,987	\$35,999	0.2418
GBT	\$52,944	\$34,507	0.2708

GBT outperformed Random Forest on all metrics with a 14.6% RMSE improvement over baseline.

6.6 Interpretation

Interpretation: The most influential features were encoded job title, location, and work type indices, confirming that role and geography are the strongest salary predictors — consistent with RDD and SQL findings.

Both models showed acceptable generalization: RF gap = 7.0%, GBT gap = 9.1%, indicating no significant overfitting.

Error Analysis: The models struggle with extreme salary outliers (e.g., \$960,000) and high-variance tech roles where salary depends heavily on company-specific factors not captured in the dataset.

6.7 Limitations

- **Methodological:** No hyperparameter tuning (grid search) was performed
- **Data:** Missing key predictors such as skills, company size, and years of experience
- **Temporal:** Dataset covers only March–April 2024, limiting generalizability
- **Noise:** High salary variance within the same job title reduces model accuracy

7. Conclusion and future work

7.1 Summary of contribution

This project delivered a complete big data analysis pipeline using Apache Spark and Scala on a real-world LinkedIn job postings dataset of 123,392 records.

Key contributions include:

- **Data Preparation:** The dataset was cleaned, reduced, and transformed through a structured four-stage preprocessing pipeline, handling missing values, salary outliers, and high-cardinality categorical features.
- **RDD Analysis:** Five distributed analyses were performed using Spark RDDs, revealing that only 12.3% of jobs are remote and that geographic location strongly influences salary levels.
- **SQL Analysis:** Five analytical queries using aggregation, window functions, and CTEs confirmed that experience level and location are the strongest salary predictors, with Executive roles averaging \$119,887.
- **Machine Learning:** Two regression models (Random Forest and GBT) were trained and evaluated. GBT achieved the best performance with $RMSE = \$52,944$ and $R^2 = 0.2708$, outperforming the baseline by 14.6%.

7.2 Reflection

This project deepened our understanding of distributed data processing using Apache Spark. Working with real-world LinkedIn data highlighted the gap between clean academic datasets and messy production data. We gained hands-on experience with the trade-offs between RDD and DataFrame APIs, and learned how preprocessing decisions directly impact model performance.

Key challenges encountered included:

- **Environment Configuration:** Setting up the Spark environment with Java 17 and managing dependencies in SBT.
- **Data Imbalance:** Dealing with highly skewed salary distributions and outliers in specific job titles.

7.3 Potential Extensions (Future Work)

To improve the model's predictive power, the following extensions are proposed:

- **Natural Language Processing (NLP):** Extracting key skills from the description column using Spark NLP.
- **Real-time Processing:** Integrating Spark Streaming to analyze job postings as they are published.
- **Advanced Ensembling:** Implementing a Stacking Regressor to combine multiple models for higher accuracy.

8. References

- [1] M. Moheez, "LinkedIn Job Listings Dataset," Hugging Face, 2024. [Online]. Available: https://huggingface.co/datasets/Moheez2611/linkedin_job_listings.
- [2] Apache Software Foundation, "Apache Spark Documentation," 2024. [Online]. Available: <https://spark.apache.org/docs/latest/>.
- [3] Apache Software Foundation, "Spark MLlib Guide," 2024. [Online]. Available: <https://spark.apache.org/docs/latest/ml-guide.html>.
- [4] Scala Documentation, "Scala Language," 2024. [Online]. Available: <https://docs.scala-lang.org/>.

9. Appendix

Appendix A: RDD Transformation Snippet

```
// -----  
// TRANSFORMATION: filter  
//  
// Filters the RDD to retain only remote jobs  
// (remote_allowed == "1.0").  
// This isolates a specific market segment that matters  
// when analyzing how work arrangement affects salary.  
// -----  
log("TRANSFORMATION: filter")  
log("Extract rows where remote_allowed = 1 (Remote Jobs Only)")  
sep()  
  
val remoteJobsRDD = splitRDD.filter(row =>  
  remoteIndex >= 0 && row.length > remoteIndex && safeGet(row, remoteIndex, "0.0") == "1.0"  
)
```

Figure 7: Filter Transformation

```
// -----  
// ACTION: count  
//  
// Counts how many remote jobs exist in the dataset.  
// -----  
val remoteCount = remoteJobsRDD.count()  
val totalCount = dataRDD.count()  
val percentage = if (totalCount > 0) remoteCount.toDouble / totalCount * 100 else 0.0  
  
log(f"Total jobs in dataset : ${totalCount},d")  
log(f"Remote jobs (count)   : ${remoteCount},d")  
log(f"Remote percentage     : ${percentage}.1f%%")  
log()
```

Figure 8: Count Action

```
// -----  
// ACTION: first  
//  
// Retrieves the first record from the filtered remote RDD  
// to validate the filter transformation.  
// -----  
if (remoteCount > 0) {  
  val firstRemote = remoteJobsRDD.first()  
  log("ACTION: first - Sample Remote Job Record")  
  log("-" * 70)  
  headerCols.zip(firstRemote).foreach { case (col, value) =>  
    log(f"  $col%-35s : $value")  
  }  
}
```

Figure 9: First Action

```
// -----  
// TRANSFORMATION: map  
//  
// Transforms each row into:  
// (state, salaryTier, workType, salary)  
//  
// This is feature engineering because it converts the raw  
// salary number into salary tiers that are easier to interpret.  
// -----  
log("TRANSFORMATION: map")  
log("Each row -> (State, SalaryTier, WorkType, Salary)")  
sep()  
  
val mappedRDD = splitRDD.map(row => {  
  val salary = parseDouble(safeGet(row, salaryIndex, "0.0"))  
  val state = safeGet(row, stateIndex)  
  val workType = safeGet(row, workTypeIndex)  
  
  val salaryTier =  
    if (salary < 60000) "Low"  
    else if (salary < 100000) "Mid"  
    else "High"  
  
  (state, salaryTier, workType, salary)  
})
```

Figure 10: : Map Transformation

```
// -----
// ACTION: take(10)
//
// Retrieves the first 10 mapped records.
// -----
log("ACTION: take(10) - Preview first 10 mapped records")
log(f"  ${"State"}%-35s ${"Tier"}%-6s ${"Work Type"}%-12s ${"Salary"}%s")
log("-" * 70)

mappedRDD.take(10).foreach { case (state, tier, workType, salary) =>
  log(f"  $state%-35s $tier%-6s $workType%-12s $$$salary%.0f")
}
```

Figure 11: Take 10 Action

```
// TRANSFORMATION: groupByKey
// Groups job titles by formatted experience level:
// (experienceLevel, title) -> groupByKey()
// -----
log("TRANSFORMATION: groupByKey")
log("Group job titles by formatted_experience_level")
sep()

val titlesByExperienceRDD = splitRDD
  .map(row => {
    val experience = cleanExperienceLevel(safeGet(row, expIndex))
    val title      = safeGet(row, titleIndex)
    (experience, title)
  })
  .groupByKey()

log("Grouped job-title preview by experience level")
log("-" * 70)

titlesByExperienceRDD
  .mapValues(titles => titles.toSet.toList.sorted)
  .collect()
  .sortBy(_._1)
  .foreach { case (experience, titles) =>
    val preview = titles.take(3).map(_.take(25)).mkString(" | ")
    log(f"  $experience%-18s -> ${titles.size}%5d unique titles | Sample: $preview")
  }
```

Figure 12: Group by Key Transformation

```
// -----
// ACTION: reduce
//
// Uses reduce to find the single record with the maximum
// normalized_salary in the dataset.
//
// This directly supports our project goal because salary
// prediction depends on identifying the upper salary bound
// and the role/location combination associated with it.
// -----
log("ACTION: reduce - Find record with maximum normalized_salary")
log("-" * 70)

val salaryRecordRDD = splitRDD.map(row => {
  val salary = parseDouble(safeGet(row, salaryIndex, "0.0"))
  val title  = safeGet(row, titleIndex)
  val state  = safeGet(row, stateIndex)
  val experience = cleanExperienceLevel(safeGet(row, expIndex))
  val workType = safeGet(row, workTypeIndex)
  (salary, title, state, experience, workType)
})

val maxSalaryRecord = salaryRecordRDD.reduce((a, b) => if (a._1 >= b._1) a else b)
val maxSalary       = maxSalaryRecord._1
```

Figure 13: Reduce Action

```
// -----
// TRANSFORMATION: reduceByKey
//
// Computes the average normalized_salary per state by
// summing salaries and counting records per state.
// This aggregation reveals geographic salary patterns
// which are directly relevant to salary prediction.
// -----
log("TRANSFORMATION: reduceByKey")
log("Compute average normalized_salary per state")
sep()

val stateSalaryRDD = splitRDD.map(row => {
  val state = safeGet(row, stateIndex)
  val salary = parseDouble(safeGet(row, salaryIndex, "0.0"))
  (state, (salary, 1))
})

val stateTotalsRDD = stateSalaryRDD.reduceByKey((a, b) =>
  (a._1 + b._1, a._2 + b._2)
)

val stateAvgRDD = stateTotalsRDD.mapValues { case (total, count) =>
  total / count
}

log("Preview: Average salary per state (top 10)")
log("-" * 70)
stateAvgRDD.take(10).foreach { case (state, avg) =>
  log(f" $state%-35s $$$avg%.0f")
}
```

Figure 14: Reduce by Key Transformation

```
// -----
// ACTION: saveAsTextFile
//
// Saves the (state, avgSalary) results to a CSV file
// using the same coalesce + rename pattern used in the
// preprocessing pipeline, producing a single clean file.
// -----
val reduceTempDir = "src/main/resources/reduceByKey_temp"
val reduceFinalPath = "src/main/resources/reduceByKey_avg_salary_by_state.csv"

stateAvgRDD
  .map { case (state, avg) => f"$state,$avg%.2f" }
  .coalesce(1)
  .saveAsTextFile(reduceTempDir)

val reduceDir = new File(reduceTempDir)
val reducePartFile = reduceDir.listFiles().filter(_.getName.startsWith("part-00000")).head
Files.move(reducePartFile.toPath, Paths.get(reduceFinalPath), StandardCopyOption.REPLACE_EXISTING)
reduceDir.listFiles().foreach(_.delete())
reduceDir.delete()
```

Figure 15: Save as Text File Action

```
// -----
// TRANSFORMATION: sortByKey
//
// Sorts the (state, avgSalary) pairs by average salary
// in descending order to rank states by compensation.
// This reveals which states offer the highest salaries,
// providing insight into geographic salary distribution.
// -----
log("TRANSFORMATION: sortByKey")
log("Sort states by average salary (descending)")
sep()

val sortedRDD = stateAvgRDD
  .map { case (state, avg) => (avg, state) }
  .sortByKey(ascending = false)
  .map { case (avg, state) => (state, avg) }

log("Top 10 states by average salary:")
log("-" * 70)
sortedRDD.take(10).foreach { case (state, avg) =>
  | log(f"  $state%-35s $$$avg%.0f")
}
```

Figure 16: Sort by Key Transformation

```
// -----
// ACTION: saveAsTextFile
//
// Saves the final sorted (state, avgSalary) results to
// a CSV file using the same coalesce + rename pattern,
// preserving the ranked compensation output as a single file.
// -----
val sortedTempDir = "src/main/resources/sortByKey_temp"
val sortedFinalPath = "src/main/resources/sortByKey_top_states_by_salary.csv"

sortedRDD
  .map { case (state, avg) => f"$state,$avg%.2f" }
  .coalesce(1)
  .saveAsTextFile(sortedTempDir)

val sortedDir = new File(sortedTempDir)
val sortedPartFile = sortedDir.listFiles().filter(_.getName.startsWith("part-00000")).head
Files.move(sortedPartFile.toPath, Paths.get(sortedFinalPath), StandardCopyOption.REPLACE_EXISTING)
sortedDir.listFiles().foreach(_.delete())
sortedDir.delete()
```

Figure 17: Save as Text File Action

Appendix B: SQL Query Snippet

```
// =====
// QUERY 1
// Question: What is the average normalized salary per experience
// level, considering only experience levels with more than 100
// job postings?
// SQL Features: GROUP BY + HAVING
// =====
log()
sep()
log(" QUERY 1: Average Salary per Experience Level (GROUP BY + HAVING)")
sep()
log("Question: What is the average normalized salary per experience")
log("level, considering only experience levels with more than 100 job postings?")
log()

val query1 = spark.sql(
  """
  |SELECT
  |  formatted_experience_level AS experience_level,
  |  ROUND(AVG(normalized_salary), 2) AS avg_salary,
  |  COUNT(*) AS job_count
  |FROM jobs
  |WHERE formatted_experience_level IS NOT NULL
  |  AND normalized_salary IS NOT NULL
  |GROUP BY formatted_experience_level
  |HAVING COUNT(*) > 100
  |ORDER BY avg_salary DESC
  |""".stripMargin)

log(f"  ${"Experience Level"}%-25s ${"Avg Salary"}%12s ${"Job Count"}%10s")
log("-" * 55)
query1.collect().foreach { row =>
  val expLevel = Option(row.get(0)).map(_.toString).getOrElse("Unknown")
  val avgSalary = Option(row.get(1)).map(_.toString).getOrElse("N/A")
  val jobCount = row.getLong(2)
  log(f"  $expLevel%-25s $avgSalary%12s $jobCount%10d")
}
```

Figure 18: Query 1

```
// =====
// QUERY 2
// Question: What are the top 10 states with the highest average
// salary, limited to states with more than 50 job postings?
// SQL Features: WHERE + GROUP BY + HAVING + ORDER BY + LIMIT
// =====
log()
sep()
log(" QUERY 2: Top 10 States by Average Salary (WHERE + ORDER BY + LIMIT)")
sep()
log("Question: What are the top 10 states with the highest average")
log("salary, limited to states with more than 50 job postings?")
log()

val query2 = spark.sql(
  """
  |SELECT
  |  state,
  |  ROUND(AVG(normalized_salary), 2) AS avg_salary,
  |  COUNT(*) AS job_count
  |FROM jobs
  |WHERE state IS NOT NULL
  |  AND TRIM(state) <> ''
  |  AND normalized_salary IS NOT NULL
  |GROUP BY state
  |HAVING COUNT(*) > 50
  |ORDER BY avg_salary DESC
  |LIMIT 10
  |""".stripMargin)

log(f"  ${"State"}%-35s ${"Avg Salary"}%12s ${"Job Count"}%10s")
log("-" * 62)
query2.collect().foreach { row =>
  val state = Option(row.get(0)).map(_.toString).getOrElse("Unknown")
  val avgSalary = Option(row.get(1)).map(_.toString).getOrElse("N/A")
  val jobCount = row.getLong(2)
  log(f"  $state%-35s $avgSalary%12s $jobCount%10d")
}
```

Figure 19: Query 2

```

// =====
// QUERY 3
// Question: How does each job's salary rank within its experience level?
// SQL Features: Window Function + RANK
// =====

log()
sep()
log(" QUERY 3: Salary Rank within Each Experience Level (WINDOW + RANK)")
sep()
log("Question: How does each job's salary rank within its experience level?")
log()

val query3 = spark.sql(
  """
  |SELECT
  |  title,
  |  formatted_experience_level AS experience_level,
  |  normalized_salary,
  |  RANK() OVER (
  |    PARTITION BY formatted_experience_level
  |    ORDER BY normalized_salary DESC
  |  ) AS salary_rank
  |FROM jobs
  |WHERE normalized_salary IS NOT NULL
  |  AND formatted_experience_level IS NOT NULL
  |ORDER BY experience_level, salary_rank
  |LIMIT 50
  |""".stripMargin)

log(s" ${"Title"%-45s} ${"Experience Level"%-20s} ${"Salary"%12s} ${"Rank"%6s}")
log("-" * 90)

query3.collect().foreach { row =>

  val title = Option(row.getAs[String]("title"))
    .map(_.replace("\n",""))
    .map(_.take(43))
    .getOrElse("Unknown")

  val exp = Option(row.getAs[String]("experience_level"))
    .getOrElse("Unknown")

  val salary = Option(row.getAs[Any]("normalized_salary")) match {
    case Some(v: Double) => s"${v%12.2f}"
    case Some(v: Float)  => s"${v%12.2f}"
    case Some(v)         => v.toString
    case None            => "N/A"
  }

  val rank = row.getAs[Int]("salary_rank")

  log(s" ${title%-45s} $exp%-20s $salary%12s $rank%6d")
}

```

Figure 20: Query 3

```

// -----
// QUERY 4
// Question: Which jobs have a salary above their state's average salary?
// SQL Features: CTE + Subquery + JOIN
// -----
log()
sep()
log(" QUERY 4: Jobs Above State Average Salary (CTE + Subquery + JOIN)")
sep()
log("Question: Which jobs have a salary above their state's average salary?")
log()

val query4 = spark.sql(
  """
  WITH state_avg AS (
    | SELECT
    |   state,
    |   ROUND(AVG(normalized_salary), 2) AS avg_state_salary
    | FROM jobs
    | WHERE state IS NOT NULL
    |   AND TRIM(state) <> ''
    |   AND normalized_salary IS NOT NULL
    | GROUP BY state
  )
  | SELECT
  |   j.title,
  |   j.state,
  |   ROUND(j.normalized_salary, 2) AS salary,
  |   sa.avg_state_salary AS state_avg_salary,
  |   ROUND(j.normalized_salary - sa.avg_state_salary, 2) AS above_avg_by
  | FROM jobs j
  | JOIN state_avg sa ON j.state = sa.state
  | WHERE j.normalized_salary > sa.avg_state_salary
  |   AND j.state IS NOT NULL
  | ORDER BY above_avg_by DESC
  | LIMIT 20
  """
  .stripMargin)

log(s" ${"Title"}X-40s ${"State"}X-15s ${"Salary"}X12s ${"State Avg"}X12s ${"Above By"}X12s")
log(".*" * 97)
query4.collect().foreach { row =>
  val title = Option(row.getAs[String]("title"))
    .map(_.replace("\n", ""))
    .map(_.take(38))
    .getOrElse("Unknown")
  val state = Option(row.getAs[String]("state"))
    .map(_.take(13))
    .getOrElse("Unknown")
  val salary = Option(row.getAs[Any]("salary")) match {
    case Some(v: Double) => s"$v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  val stateAvg = Option(row.getAs[Any]("state_avg_salary")) match {
    case Some(v: Double) => s"$v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  val aboveBy = Option(row.getAs[Any]("above_avg_by")) match {
    case Some(v: Double) => s"$v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  log(s" $titleX-40s $stateX-15s $salaryX12s $stateAvgX12s $aboveByX12s")
}

```

Figure 21: Query 4

```
// =====
// QUERY 5
// Question: What is the salary distribution across work types,
// including average, variance, percentiles, and distinct job title count?
// SQL Features: Statistical Summary (AVG, VAR_POP, PERCENTILE_APPROX,
// COUNT DISTINCT, STDDEV_POP)
// =====
log()
sep()
log("QUERY 5: Salary Distribution by Work Type (Statistical Summary)")
sep()
log("Question: What is the salary distribution across work types,")
log("including average, variance, percentiles, and distinct job title count?")
log()

val query5 = spark.sql(
  """
  |SELECT
  | formatted_work_type                                AS work_type,
  | COUNT(*)                                           AS total_jobs,
  | COUNT(DISTINCT title)                             AS distinct_titles,
  | ROUND(AVG(normalized_salary), 2)                  AS avg_salary,
  | ROUND(STDDEV_POP(normalized_salary), 2)           AS salary_stddev,
  | ROUND(VAR_POP(normalized_salary), 2)              AS salary_variance,
  | ROUND(PERCENTILE_APPROX(normalized_salary, 0.25), 2) AS p25_salary,
  | ROUND(PERCENTILE_APPROX(normalized_salary, 0.50), 2) AS median_salary,
  | ROUND(PERCENTILE_APPROX(normalized_salary, 0.75), 2) AS p75_salary
  |FROM jobs
  |WHERE formatted_work_type IS NOT NULL
  | AND normalized_salary IS NOT NULL
  |GROUP BY formatted_work_type
  |ORDER BY avg_salary DESC
  |""".stripMargin)

log(s"Work Type" % 15s s"Jobs" % 10s s"Titles" % 10s s"Avg Salary" % 12s s"Std Dev" % 12s s"P25" % 10s s"Median" % 10s s"P75" % 10s)
log(" " * 100)
query5.collect().foreach { row =>
  val workType = Option(row.getAs[String]("work_type")).getOrElse("Unknown")
  val totalJobs = row.getLong(1)
  val distinctTitles = row.getLong(2)
  val avgSalary = Option(row.getAs[Any]("avg_salary")) match {
    case Some(v: Double) => s"%v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  val stddev = Option(row.getAs[Any]("salary_stddev")) match {
    case Some(v: Double) => s"%v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  val p25 = Option(row.getAs[Any]("p25_salary")) match {
    case Some(v: Double) => s"%v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  val median = Option(row.getAs[Any]("median_salary")) match {
    case Some(v: Double) => s"%v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  val p75 = Option(row.getAs[Any]("p75_salary")) match {
    case Some(v: Double) => s"%v%.2f"
    case Some(v)         => v.toString
    case None            => "N/A"
  }
  log(s"WorkType: %15s TotalJobs: %10d DistinctTitles: %10d AvgSalary: %12s StdDev: %12s P25: %10s Median: %10s P75: %10s")
}
```

Figure 22: Query 5

Appendix C: Machine Learning Operations Snippet

```
// =====
// BASELINE MODEL: MEAN PREDICTION
// =====

sep()
log(" BASELINE MODEL: Mean Salary Prediction")
sep()

// Compute mean salary on training set only (no leakage)
val meanSalary = trainTransformed
  .agg(avg("normalized_salary"))
  .collect()(0)
  .getDouble(0)

log(s" Mean salary (train): ${meanSalary}%.2f")

val baselinePredictions = testTransformed
  .withColumn("prediction", lit(meanSalary))

val baselineRMSE = evaluatorRMSE.evaluate(baselinePredictions)
val baselineMAE = evaluatorMAE.evaluate(baselinePredictions)
val baselineR2 = evaluatorR2.evaluate(baselinePredictions)

log(s" Baseline RMSE : ${baselineRMSE}%.2f")
log(s" Baseline MAE : ${baselineMAE}%.2f")
log(s" Baseline R2 : ${baselineR2}%.4f")

sep()
log(" BASELINE COMPLETE")
sep()
```

Figure 23: Baseline Model

```

// =====
// MODEL 1: RANDOM FOREST REGRESSOR TRAIN & TEST METRICS
// =====

sep()
log(" MODEL 1: Random Forest Regressor Full Evaluation")
sep()

val rf = new RandomForestRegressor()
    .setLabelCol("normalized_salary")
    .setFeaturesCol("features")
    .setPredictionCol("prediction")
    .setNumTrees(50)
    .setMaxDepth(10)

val rfModel = rf.fit(trainTransformed)

// --- Training metrics (detect overfitting) ---
log("\n----- RF TRAINING METRICS -----")
val rfTrainPred = rfModel.transform(trainTransformed)
val rfTrainRMSE = evaluatorRMSE.evaluate(rfTrainPred)
val rfTrainMAE = evaluatorMAE.evaluate(rfTrainPred)
val rfTrainR2 = evaluatorR2.evaluate(rfTrainPred)

log($" Train RMSE : ${rfTrainRMSE%.2f}")
log($" Train MAE : ${rfTrainMAE%.2f}")
log($" Train R2 : ${rfTrainR2%.4f}")

// --- Test metrics ---
log("\n----- RF TEST METRICS -----")
val rfTestPred = rfModel.transform(testTransformed)
val rfTestRMSE = evaluatorRMSE.evaluate(rfTestPred)
val rfTestMAE = evaluatorMAE.evaluate(rfTestPred)
val rfTestR2 = evaluatorR2.evaluate(rfTestPred)

log($" Test RMSE : ${rfTestRMSE%.2f}")
log($" Test MAE : ${rfTestMAE%.2f}")
log($" Test R2 : ${rfTestR2%.4f}")

// --- Overfitting check ---
log("\n----- RF OVERFITTING CHECK -----")
val rfRmseGap = ((rfTestRMSE - rfTrainRMSE) / rfTrainRMSE) * 100
log($" RMSE gap (test-train) / train: ${rfRmseGap%.1f}%")
if (rfRmseGap > 20) log(" WARNING: Possible overfitting detected")
else log(" OK: Generalisation gap is within acceptable range")

// --- vs Baseline ---
log("\n----- RF vs BASELINE -----")
val rfImprovement = ((baselineRMSE - rfTestRMSE) / baselineRMSE) * 100
log($" Baseline RMSE : ${baselineRMSE%.2f}")
log($" RF Test RMSE : ${rfTestRMSE%.2f}")
log($" RMSE improvement over baseline: ${rfImprovement%.1f}%")

// --- Feature Importances (top 5) ---
log("\n----- RF TOP FEATURE IMPORTANCES -----")
val importances = rfModel.featureImportances.toArray
val top5 = importances.zipWithIndex.sortBy(_._1).take(5)
top5.foreach { case (imp, idx) =>
  log($" Feature[${idx}] : ${imp%.6f}")
}

```

Figure 24: Model 1

```
// =====
// MODEL 2: GRADIENT BOOSTED TREES ▢ TRAIN & TEST METRICS
// =====

sep()
log(" MODEL 2: Gradient Boosted Trees (GBT) ▢ Full Evaluation")
sep()

val gbt = new GBRegressor()
  .setLabelCol("normalized_salary")
  .setFeaturesCol("Features")
  .setPredictionCol("prediction")
  .setMaxIter(100)
  .setMaxDepth(5)

val gbtModel = gbt.fit(trainTransformed)

// --- Training metrics ---
log("\n----- GBT TRAINING METRICS -----")
val gbtTrainPred = gbtModel.transform(trainTransformed)
val gbtTrainRMSE = evaluatorRMSE.evaluate(gbtTrainPred)
val gbtTrainMAE = evaluatorMAE.evaluate(gbtTrainPred)
val gbtTrainR2 = evaluatorR2.evaluate(gbtTrainPred)

log(f" Train RMSE      : ${gbtTrainRMSE}%.2f")
log(f" Train MAE       : ${gbtTrainMAE}%.2f")
log(f" Train R2        : ${gbtTrainR2}%.4f")

// --- Test metrics ---
log("\n----- GBT TEST METRICS -----")
val gbtTestPred = gbtModel.transform(testTransformed)
val gbtTestRMSE = evaluatorRMSE.evaluate(gbtTestPred)
val gbtTestMAE = evaluatorMAE.evaluate(gbtTestPred)
val gbtTestR2 = evaluatorR2.evaluate(gbtTestPred)

log(f" Test RMSE       : ${gbtTestRMSE}%.2f")
log(f" Test MAE        : ${gbtTestMAE}%.2f")
log(f" Test R2        : ${gbtTestR2}%.4f")

// --- Overfitting check ---
log("\n----- GBT OVERFITTING CHECK -----")
val gbtRmseGap = ((gbtTestRMSE - gbtTrainRMSE) / gbtTrainRMSE) * 100
log(f" RMSE gap (test-train) / train: ${gbtRmseGap}%.1f%%")
if (gbtRmseGap > 20) log(" WARNING: Possible over-fitting detected")
else log(" OK: Generalisation gap is within acceptable range")

// --- vs Baseline ---
log("\n----- GBT vs BASELINE -----")
val gbtImprovement = ((baselineRMSE - gbtTestRMSE) / baselineRMSE) * 100
log(f" Baseline RMSE : ${baselineRMSE}%.2f")
log(f" GBT Test RMSE : ${gbtTestRMSE}%.2f")
log(f" RMSE improvement over baseline: ${gbtImprovement}%.1f%%")

// --- Feature Importances (top 5) ---
log("\n----- GBT TOP FEATURE IMPORTANCES -----")
val gbtImportances = gbtModel.featureImportances.toArray
val gbtTop5 = gbtImportances.zipWithIndex.sortBy(_._1).take(5)
gbtTop5.foreach { case (imp, idx) =>
  log(f" Feature[${idx}] : ${imp}%.6f")
}
```

Figure 25: Model 2

```
// =====
// FINAL COMPARISON SUMMARY
// =====

sep()
log(" FINAL MODEL COMPARISON SUMMARY")
sep()

log("\n" + "%-20s %-15s %-15s %-10s".format("Model", "RMSE (test)", "MAE (test)", "R2 (test)"))
sub()
log("%-20s %-15s %-15s %-10s".format(
  "Baseline (mean)",
  f"${baselineRMSE}%.0f",
  f"${baselineMAE}%.0f",
  f"${baselineR2}%.4f"))
log("%-20s %-15s %-15s %-10s".format(
  "Random Forest",
  f"${rFTestRMSE}%.0f",
  f"${rFTestMAE}%.0f",
  f"${rFTestR2}%.4f"))
log("%-20s %-15s %-15s %-10s".format(
  "GBT",
  f"${gbtTestRMSE}%.0f",
  f"${gbtTestMAE}%.0f",
  f"${gbtTestR2}%.4f"))
sub()

if (gbtTestRMSE < rFTestRMSE)
  log(" WINNER: GBT outperforms Random Forest on test RMSE")
else
  log(" WINNER: Random Forest outperforms GBT on test RMSE")

sep()
log(" EVALUATION COMPLETE")
sep()
```

Figure 26: Comparison